Progreso del curso · 32 capítulos, 9 partes

Parte 0
Introducción

1 2

Parte I
Fundamentos estructurales y de datos

3 4 5◇ 6 7 8 9

Parte II
Estimación flexible y primer despliegue

10 11 12 13 14 **15**

Parte III
Diseños de identificación con ML

16◇ 17◇ 18◇

Parte IV
Publicación y operación en producción

19 20

Parte V
Heterogeneidad, políticas y targeting

21 22

Parte VI
Explicación, equidad y decisión

23 24 25 26

Parte VII
Métodos avanzados

27◇ 28◇

Parte VIII
Estructura, monitoreo y frontera

29 30 31 32

■ previos · ■ actual · □ próximos · ◇ profundización electiva (no bloquea el camino ⋆)

15

CAPÍTULO 15

★ Camino crítico

Despliegue de un estimador causal como servicio

Parte II · Sesión S12

Plantilla B (ingeniería y producción)

IA Causal en Producción

Curso ECO-490 / ECON-540 · Edición 2026

Roadmap del capítulo

Objetivos de aprendizaje

1. Justificar biblioteca vs. servicio vs. batch para un caso dado.
2. Separar la función de inferencia pura del framework web.
3. Explicar qué expone /metadata que /health no expone.

⋆ Antes de clase

Lectura: Cap. 15 — Despliegue como servicio (@cap-despliegue-servicio)
Lab: L12
Entrega/hito: B1 arranca · Validar con grade_b1.py

Agenda

01 · Cómo poner el estimador a disposición de otros

02 · Biblioteca / servicio REST / batch precomputado

03 · Decisión: FastAPI + Docker

04 · inference.py puro + app.py + /metadata

05 · Por qué la respuesta incluye IC y población de entrenamiento

El problema de diseño

Decisión	Servir el estimador como un servicio REST síncrono, no como biblioteca embebida ni como batch.
Alternativas	Biblioteca embebida vs. servicio web vs. predicciones batch precomputadas.
Trade-off clave	Latencia y simplicidad de biblioteca vs. desacoplamiento y reutilización de un servicio.
Stack	FastAPI, Docker, joblib.
Riesgo principal	Training-serving skew: que el cálculo de features en producción difiera del usado al entrenar.

Opciones y trade-offs

Biblioteca embebida
Menor latencia, más simple de depurar. Cada consumidor necesita el mismo entorno; actualizar exige redistribuir.

Servicio REST/gRPC
Consumidores en cualquier lenguaje; un solo lugar para actualizar. Costo: latencia de red y operar el servicio.

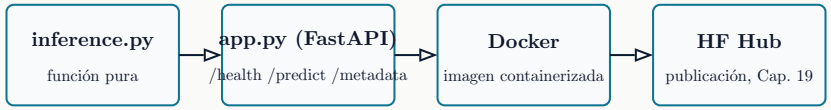
Batch precomputado
Menor latencia de consulta, más auditable. No sirve para unidades nuevas ni covariables que cambiaron desde el cómputo.

Decisión adoptada

Servicio REST síncrono con FastAPI, containerizado con Docker. La función de inferencia (inference.py) se separa del framework web (app.py) y reutiliza el CausalSchema del Cap. 4.

Publicar en Hugging Face Hub (Cap. 19) requiere que cualquiera lo consulte sin instalar el entorno del curso; los Notebooks A2/A4 requieren estimar para unidades nuevas, no solo una población fija.

Arquitectura del servicio: de la función pura al Hub



inference.py — función pura

```python
# inference.py -- funcion pura, sin dependencias de FastAPI
import joblib
import pandas as pd

_modelo = joblib.load('model/dml_estimator.joblib')
_schema = joblib.load('model/causal_schema.joblib')

def estimar_efecto(covariables: dict) -> dict:
    """Funcion pura: mismas entradas producen la misma salida,
    sin efectos secundarios. Se prueba igual en notebook y en produccion."""
    faltantes = set(_schema.covariates) - set(covariables.keys())
    if faltantes:
        raise ValueError(f'Faltan covariables requeridas: {faltantes}')
    X = pd.DataFrame([covariables])[_schema.covariates]
    efecto = _modelo.effect(X)[0]
    ic_inf, ic_sup = _modelo.effect_interval(X, alpha=0.05)
    return {
        'efecto_estimado': float(efecto),
        'ic_95_inferior': float(ic_inf[0]),
        'ic_95_superior': float(ic_sup[0]),
        # ic_individual: solo si hay calibracion conforme (Cap. 14)
        'poblacion_entrenamiento': _schema.covariates,  # transparencia
    }
```

app.py — /health, /metadata, /predict

```python
# app.py -- FastAPI delgado alrededor de inference.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from inference import estimar_efecto, metadatos_modelo

app = FastAPI(title='Servicio de estimacion causal')

class Covariables(BaseModel):
    exper: float; black: int; south: int; married: int

@app.post('/predict')
def predict(cov: Covariables):
    try:
        return estimar_efecto(cov.model_dump())
    except ValueError as e:
        raise HTTPException(status_code=422, detail=str(e))

@app.get('/health')
def health():
    return {'status': 'ok'}  # liveness; no valida el contrato causal

@app.get('/metadata')
def metadata():
    # Contrato causal legible por maquina: estimando, supuestos, poblacion,
    # version, hash_datos_entrenamiento; valida contra
    # causal-model-card-v1.schema.json (Ap. G.4)
    return metadatos_modelo()
```

💡 El orden de la prueba de humo es health → metadata → predict: un fallo en metadata con éxito en health es el escenario silencioso que este capítulo hace visible.

02_monografia → 03_notebooks_caso_estudio / 04_notebooks_produccion_hf / 05_labs_sesion

04_notebooks_produccion_hf/B1_fastapi_dml/ (FastAPI, inference.py pura, Docker). Artefacto de entrada: artifacts/A2/dml_estimator.joblib + feature_list.json. Labs: L12, L14. Humo: autograder/grade_b1.py.

Recordatorio del caso

El Cap. 2 comprometió que la interfaz debe comunicar algo más que un número.

Resultado numérico

estimar_efecto() devuelve efecto_estimado, ic_95_inferior, ic_95_superior, poblacion_entrenamiento — no solo el punto.

Lectura de política · Ningún consumidor del servicio puede usar el efecto fuera de su población sin, al menos, tener esa información disponible en la propia respuesta.

Nivel PCH	**Estimando / métrica principal**
Transversal (sirve una estimación de L2 ya validada)	/predict con efecto + IC; /metadata con contrato causal legible por máquina

Pregunta activa (1 min)

Si /health responde 200 OK pero /metadata expone un hash de datos distinto al esperado, ¿el sistema está "arriba"?

Escalera PCH: L1 (asociación) ⊆ L2 (intervención) ⊆ L3 (contrafactual) — cada nivel exige supuestos estrictamente más fuertes que el anterior. Si solo predijéramos, ¿a quién trataríamos? Con el efecto, ¿cambia la decisión?

Señales de alerta y checklist operativo

- ☐ La función de inferencia calcula features con código distinto al de entrenamiento.
- ☐ El endpoint devuelve solo el efecto puntual, sin IC ni población.
- ☐ No existe una ruta /health verificable independientemente de /predict.
- ☐ No hay forma de saber, consultando el servicio, qué versión del modelo responde ni sobre qué datos se entrenó.

Si dos o más señales están presentes, el sistema no está listo según los criterios de este curso.

Punto de quiebre · Aplicado

★ ¿Por qué separar inference.py de app.py facilita probar la lógica de inferencia sin levantar el servidor web?

⏱️ 2-4 min · Votación mano alzada o Mentimeter

Discusión y ejercicio

Pregunta de discusión

Para el caso del Notebook A4 (política óptima, número fijo de clientes cada mes), ¿cuál de las tres opciones sería más apropiada? Justifique.

Ejercicio (5-10 min)

Complete el Notebook B1: serialice el estimador del Notebook A2, adapte los tres listados, construya la imagen Docker y verifique /health + /metadata + /predict con curl.

Cierre del capítulo

Resumen

- La decisión de despliegue (biblioteca/servicio/batch) depende del caso de uso, no hay opción universalmente correcta.
- inference.py puro + app.py delgado evita training-serving skew y facilita testear la lógica sin servidor.
- /metadata expone el contrato causal en tiempo de ejecución — versión, supuestos, población — legible por máquina.

Referencias clave

(1) Huyen (2022), Cap. 7, «Model Deployment and Prediction Service»

(2) Kästner, Kang & Hilton (2024), Cap. 10, «Deploying a Model»

Próximo episodio · Cap. 16 — Post-Double-Selection LASSO (◇, ampliación de S5; en el libro cierra la Parte II)

Gancho: El servicio ya expone un ATE; la Parte III retoma la pregunta de identificación con diseños completos, empezando por la que ya se usó en S5 en su versión avanzada.